

Scaffolding Over Scale: Architecture as the Driver of Deployed Agent Capability

A Falsifiable Protocol and a Self-Constructing System Case Study

Andres G. and the PaxLabs Matrix Team

PaxLabs Inc.

Correspondence: legal@Paxeer.app

Reference implementation: <https://github.com/paxlabs-inc/matrix-core>

Abstract - The prevailing account attributes recent gains in artificial intelligence to the scaling of large language models: more parameters, more tokens, and more compute. This paper advances a narrower claim about deployed, multi-step, side-effecting agent capability. Once a model crosses a practical competence threshold, the marginal capability that reaches users is mediated primarily by the architecture around the model rather than by model size alone. We call this architecture the scaffolding stack and characterize four primitives: a typed intermediate representation of intent, a persistent typed memory substrate, deterministic reproducible execution, and verifiable provenance. We state the thesis as a falsifiable comparison of capability gradients under controlled conditions, and we propose two tests: a scaffolding ablation that holds the model fixed while removing architectural primitives, and a model-swap sweep that holds the full stack fixed while varying model scale and vendor. As construction-time evidence, we present a reference system in which a small human team hand-authored the memory substrate and a machine-readable specification, after which substrate-bound agents produced the remaining application modules. The case study is not presented as proof of a universal law. It is an existence proof that a specification-bound, provenance-heavy agent architecture can support a long-horizon engineering build, and it motivates the independent evaluation protocol. We close with the research position that progress toward more general artificial intelligence is likely to depend at least as much on architecture, memory, reproducibility, and repair as on further model scaling.

Index Terms - agentic systems, intermediate representation, neuro-symbolic compilation, reproducibility, provenance, scaling laws, typed memory, AGI

I. Introduction

The dominant explanation for progress in artificial intelligence is scale. Capability is often treated as a smooth function of parameter count, training tokens, and compute. That account is valuable for modeling training loss and raw model behavior, but it is incomplete for the quantity practitioners care about most: whether an AI system can reliably complete a multi-step task, use tools safely, preserve intent across time, and leave a record that humans can inspect, repair, and trust.

This paper challenges the scale-first account at the level of deployed agent capability. We do not argue that scaling laws are wrong, that larger models are useless, or that architecture can turn an incompetent model into a competent one. The claim is narrower: for real agentic systems that act through tools, memory, plans, and side effects, the surrounding architecture increasingly determines how much model competence becomes useful capability.

We therefore advance the following research position rather than a settled result: for deployed, multi-step, side-effecting AI systems, the marginal advance of useful capability is now driven primarily by agentic scaffolding - the architecture surrounding a model - once the underlying model exceeds a practical competence threshold. The road to more general artificial intelligence is therefore likely to depend on architecture, memory, reproducibility, and repair, not only on larger models.

The contribution of this paper is not the assertion that one implementation proves the thesis. It does not. The contribution is a concrete account of the architectural primitives, a falsifiable evaluation protocol, and a case study showing that the architecture-heavy route can produce a large working artifact under a small human budget.

A. Contributions

- **A falsifiable position.** We formalize architecture over scale as a comparison of capability gradients under a fixed task distribution and environment.

- **Four architectural primitives.** We identify the primitives that convert a model into a reliable agent: typed intent IR, persistent typed memory, deterministic execution, and verifiable provenance.
- **A controlled evaluation protocol.** We propose a scaffolding ablation and a model-swap sweep that estimate the relative contributions of architecture and scale.
- **A construction-time case study.** We report a self-constructing reference system as an existence proof, while stating the limits of what such a case study can establish.
- **Clear claim boundaries.** We separate evidence, existence proof, and proof of thesis to make the paper harder to dismiss as marketing or self-report.

B. Claims Not Made

- We do not claim that model scaling is irrelevant. Larger and better-trained models can improve raw competence and reduce error rates.
- We do not claim that architecture can replace model capability. A minimum competence threshold is required before scaffolding can carry useful work.
- We do not claim that the Matrix case study proves a universal law. It is evidence consistent with the thesis and a reason to run the protocol, not a substitute for the protocol.
- We do not claim that agents designed the system from nothing. Human design intelligence lives in the substrate and specification. The claim is that agents executed a large build while bound to that architecture.

II. The Architecture-Over-Scale Thesis

A. Where the scale account strains

Compute-optimal scaling is an excellent account of training loss as a function of parameters and tokens. It is a weaker account of deployed capability because deployed capability is mediated by the interface between a user's words and the system's actions. In agentic systems, the interface includes representations, tool protocols, memory, execution machinery, provenance, permissions, and repair loops.

Four failure modes dominate that mediation layer. Prompt sensitivity occurs when small rephrasings produce divergent behavior. Semantic drift occurs when original intent degrades across context windows, tool calls, and sub-agents. An absent shared ontology leaves the human and the agent operating over different conceptual models. Finally, without structured repair, the user has to restart or edit raw prompts instead of inspecting and amending a typed object.

A larger model can lower the rate of these failures, but it does not eliminate the class. Each failure is rooted in an untyped, non-reproducible, weakly auditable interface. Model scale can improve inference inside that interface; architecture changes the interface itself.

B. The thesis stated precisely

Let C denote deployed task-completion capability on a fixed task distribution and environment. Let M denote the model layer, including parameters, training data, decoding, and vendor behavior. Let S denote the scaffolding layer, including representation, memory, execution, provenance, permissioning, and repair. The scale-first account treats C as primarily a function of M with S held approximately constant. We claim that, in the present regime for multi-step side-effecting tasks, improvements to S often produce larger marginal gains in C than equivalent improvements to M after a model competence threshold has been reached.

Operationally, the thesis is not a slogan. Hold S fixed and vary M . Hold M fixed and vary S . Compare the changes in task completion, reproducibility, intent fidelity, repair cost, and replay integrity. If removing scaffolding causes a large drop while increasing model scale under the full stack causes a smaller gain, the task distribution supports the thesis. If model scale dominates under the same controls, the thesis is weakened or refuted for that distribution.

III. The Four Architectural Primitives

The scaffolding stack converts prose into executable, inspectable, and replayable work. Natural language is lowered to a typed IR, the IR is planned and executed against persistent memory, and every artifact is bound to provenance. The primitives are individually familiar; the claim is that their composition is now a major source of deployed capability.

P1. A typed intermediate representation of intent

The first primitive is a typed intermediate representation that sits between natural language and execution. The model performs semantic parsing into a symbolic object, and downstream stages execute that object deterministically. The IR must have a closed operation vocabulary and typed operands so classification is bounded rather than open-ended. Bounding the vocabulary at the syntax level is the structural antidote to prompt sensitivity: meaning is typed once and then carried forward as data, not re-inferred at every step.

The IR should also be content-addressed. Canonical serialization and hashing give the intent an identifier. Any mutation becomes visible, and semantically identical programs can be recognized across runs. This makes the IR analogous to a compiler contract between front-end understanding and back-end execution.

P2. A persistent typed memory substrate

The second primitive is persistent memory represented as a typed graph with an append-only journal. The journal is authoritative; indices, embeddings, summaries, and derived views are rebuildable. A bounded taxonomy of record types - identity, facts, preferences, goals, constraints, events, capabilities, and learned patterns - gives memory a schema instead of reducing it to a text blob.

Persistent typed memory supplies what context windows do not: durable, auditable state across episodes. Context length can make more information available in a single call, but it does not by itself create a stable identity, an event history, or a principled revision mechanism. For long-horizon agents, memory is not a convenience. It is the substrate on which continuity, repair, and responsibility depend.

P3. Deterministic reproducible execution

The third primitive is an execution layer whose behavior can be replayed. A planner lowers the typed IR into an explicit plan containing sequential steps, parallel branches, tool invocations, sub-agent dispatches, and human-decision gates. The plan is validated against structural invariants before execution. Runs are seeded from a digest of inputs such as intent, actor, memory snapshot, program version, model identifier, tool versions, and decoding parameters.

The purpose is not to pretend that language models are inherently deterministic. The purpose is to bind nondeterminism to recorded inputs so that a run can be explained, compared, and replayed to the greatest practical extent. Determinism converts the statement "the model did something" into "this artifact was produced from these inputs under these constraints." That distinction is the foundation of debugging and trust.

P4. Verifiable provenance

The fourth primitive binds the stack together. Every artifact is content-addressed. Every state transition is journaled and signed. Derived state is reconstructible from the journal. A replay invariant requires that, after dropping derived indices and re-walking the journal, the system reproduces the same canonical root. Divergence is treated as a defect rather than an accepted cost of agent behavior.

In a system where machine-produced work may outnumber human-authored work, provenance is the mechanism that makes output reviewable and ownable. It replaces vague confidence with inspectable history: what was requested, what specification governed it, which model and tool versions participated, which artifacts resulted, and which human approvals or repairs occurred.

IV. Evaluation Design

The architecture-over-scale thesis is falsifiable. The following protocol is the core empirical contribution. It estimates the effect of scaffolding and model scale on the same task distribution rather than relying on anecdotes or one build story.

A. Task suite

Construct a suite of multi-step tasks expressed in prose. Each task must include an objective success predicate, a partial-credit rubric, and a side-effect classification. The suite should include at least the following categories:

- Code modification with tests, where success is measured by test pass rate, linting, and intended behavior.
- Multi-tool business workflow, where success requires correct tool choice, parameter construction, and state update.
- Memory-dependent task, where success depends on retrieving and applying durable facts from earlier episodes.

- Reversible low-stakes task, where repair can happen after execution with little cost.
- Irreversible or high-stakes task, where the correct behavior includes human gates, policy checks, and refusal when execution would be unsafe or unauthorized.

B. Metrics

- **Task completion rate.** The fraction of tasks whose success predicate holds.
- **Intent fidelity.** The divergence between stated user intent and the executed plan, scored against the rubric.
- **Reproducibility.** Agreement of outcomes, canonical IR hashes, and plan hashes across repeated runs at fixed inputs.
- **Repair cost.** The number and complexity of structured corrections required to fix a wrong plan compared with restarting from a prompt.
- **Replay integrity.** The fraction of runs whose journal rebuilds to the same canonical root.
- **Safety gate correctness.** The rate at which the system inserts, preserves, or escalates required human-decision gates in high-stakes tasks.
- **Audit completeness.** The fraction of outputs with sufficient provenance to identify the governing spec, input digest, model/tool versions, and approval path.

C. Experiment 1: scaffolding ablation

Hold the model, tools, task suite, and environment fixed. Remove primitives one at a time and measure every metric:

- Full stack: typed IR, typed memory, deterministic execution, and provenance.
- Weakened P1: open-ended classification instead of a closed vocabulary.
- Removed P1: free-form plans instead of a typed IR.
- Removed P2: stateless prompting instead of persistent typed memory.
- Removed P3 and P4: no seed binding, no replay invariant, and incomplete provenance.
- Bare baseline: a ReAct-style loop using the same model and tools.

Prediction: completion rate, reproducibility, intent fidelity, repair cost, and audit completeness degrade as primitives are removed. The largest drop is expected at the typed-IR and closed-vocabulary boundary because that boundary determines whether intent is represented as durable data or continuously reinterpreted prose.

D. Experiment 2: model-swap sweep

Hold the full stack fixed and sweep models across vendors and sizes. Include small, medium, frontier, and reasoning-optimized models. Use the same tool environment, decoding budget, memory snapshot, and task suite. Above a practical competence threshold, the prediction is that completion rate and auditability are comparatively flat across scale because the architecture absorbs much of the variance. Below that threshold, the model may fail to produce valid IR or plans, and the stack cannot compensate.

E. Decision rule

Large degradation from removing scaffolding, together with comparatively smaller gains from increasing model scale under the full stack, supports the thesis for the task distribution. The opposite result refutes or weakens the thesis. A mixed result is still informative: it identifies which task families are architecture-limited and which remain model-limited.

V. Existence Proof: A Self-Constructing System

The protocol above tests the thesis on controlled tasks. This section reports a construction-time observation at the opposite extreme: a complete engineering project in which the build process attempted to isolate the architecture variable. The system is presented as a case study, not as proof of a universal law.

A. The construction as a natural experiment

A small human team authored the persistent typed memory substrate and a complete machine-readable specification. The specification was committed into the substrate. An agent population, each bound to that substrate and constrained by the specification, then produced the surrounding application modules: the intent compiler, executor, plan validator,

provenance and replay machinery, services, and tests. The resulting application layer is approximately 312,000 lines across twelve independently buildable modules.

The important distinction is design versus construction. The human-authored specification encodes substantial design intelligence. The agents did not invent the architecture from nothing. The case study instead shows that, once design is represented as machine-readable constraints inside a durable substrate, construction can parallelize across agents while preserving shared invariants.

B. Measured scale of the reference implementation

Quantity	Value
Application code, Go/Solidity/TS/JS/Python	approx. 312,000 LOC
Go	134,598 LOC
Solidity	75,799 LOC
TypeScript/TSX	51,473 LOC
JavaScript	38,634 LOC
Python	11,620 LOC
Declarative compiler language	19,327 LOC
Independently buildable modules	12
Test files	163
Tracked files	approx. 2,644

C. What the artifact supports

The artifact supports two narrower claims. First, it shows scale under a fixed human budget: a small team can produce and maintain a multi-language system when the architecture turns specification into executable work across agents. Second, it shows invariant preservation across a large corpus: version-pinned references, replay gates, provenance records, and typed memory constraints can be enforced uniformly enough to reduce observable drift across key system boundaries.

These observations are consistent with the architecture-over-scale thesis because the variable that grew was not a larger model but a stronger substrate, a more complete specification, and a population of agents bound to that substrate. They do not, by themselves, prove that architecture dominates scale in all tasks.

D. Audit bundle for independent inspection

To make the construction claim reviewable, each material artifact should be accompanied by a provenance record containing:

- the governing specification section and version;
- the actor or agent identity that produced the artifact;
- the model identifier, tool versions, decoding parameters, and execution seed when available;
- the input digest, output digest, and content-addressed artifact hash;
- the CI job, test result, replay result, and invariant checks;
- the human review or approval disposition; and
- the repair record if the artifact was amended after generation.

This audit bundle does not eliminate all self-report risk, but it changes the claim from "trust us" to "inspect the chain." A reviewer can sample artifacts, replay derived state, inspect provenance records, and verify that claimed invariants are enforced by the build and test system.

E. Cross-model construction

The end-to-end harness should exercise the full stack across multiple model providers. The relevant test is not whether every model produces identical raw text. The relevant test is whether a valid specification compiles into equivalent

typed artifacts, executable plans, and replayable records across model substitutions. If the full stack produces working executions across vendors while the bare loop degrades sharply, that pattern is exactly what the thesis predicts.

F. Threats to validity

- **Self-report.** The claim about who wrote what is only as strong as the provenance trail. The mitigation is to publish or provide an audit bundle, CI logs, replay artifacts, and artifact hashes.
- **Specification confound.** The human-authored specification contains substantial design intelligence. This is not a weakness of the thesis; it is the thesis. Capability lives in the substrate and specification as much as in the model.
- **Generalization.** One artifact cannot establish a universal law. The controlled protocol in Section IV is required to test the claim on other task distributions.
- **Survivorship bias.** A successful build may hide failed generations, retries, or human repairs. The audit bundle should therefore include repair records and rejected artifacts where possible.
- **Repository availability.** If the reference implementation is not publicly accessible at review time, claims that depend on independent inspection should be treated as provisional until the relevant evidence package is available.

VI. Related Work

Scaling laws. Kaplan et al. and Hoffmann et al. model loss as a function of scale and compute. This paper does not dispute those results. It argues that training loss is not the same object as deployed multi-step capability.

Tool use and agentic reasoning. Toolformer shows that tool augmentation can change what a model can accomplish. ReAct shows that interleaving reasoning and acting improves task performance. Surveys of autonomous agents catalog the expanding design space. This paper differs by treating intent itself as a typed, content-addressed, reproducible object rather than an ephemeral trace.

Small models and post-scale architecture. Recent work argues that small language models are often sufficient and economically preferable for many agentic invocations. Other work frames the missing ingredient for general intelligence as coordination structure. This paper supplies a concrete characterization of that structure and a protocol to test its causal weight.

Reproducibility of agents. Recent measurements of behavioral drift in tool-calling pipelines motivate deterministic execution and provenance. The proposed stack treats drift not as a mysterious property of agents but as a defect that should be bounded, logged, and replay-tested.

Intermediate representations and event sourcing. Compiler IRs, event-sourced stores, signed journals, and Merkle-style provenance are standard techniques. The novelty claimed here is not any one technique in isolation; it is the composition of these techniques into an agent architecture for reliable, repairable, long-horizon work.

VII. Discussion: Architecture as an AGI Research Program

We restate the AGI claim carefully. The paper does not prove that architecture is the only road to general intelligence, nor that larger models will stop mattering. It argues that the properties required for general agency - durable identity, structured memory, tool-grounded action, reproducible execution, permissioned side effects, and repair - are architectural properties.

Generality is composition. A closed operation vocabulary plus an expanding marketplace of skills and tools yields open-ended behavior without open-ended classification at every step. Capability can grow by adding typed operations, tools, memory structures, and validators.

Memory is identity. A persistent, typed, replayable per-agent store gives an agent continuity over time. Context-window scaling can place more text in a prompt, but it does not by itself create a durable self, a revision history, or an auditable belief graph.

Repair is intelligence. The ability to inspect a structured plan, amend it mid-flight, preserve provenance, and continue execution is closer to practical general agency than one-shot generation. A system that cannot be repaired can be impressive without being dependable.

The practical conclusion is that the next capability frontier is not only better models. It is better scaffolding: typed intent, persistent memory, deterministic execution, provenance, permissions, and repair loops that turn model competence into owned work.

VIII. Conclusion

This paper advanced the position that agentic scaffolding now drives a large share of deployed AI capability for multi-step, side-effecting systems. The claim was stated as falsifiable rather than asserted as settled fact. We characterized the scaffolding stack with four primitives: typed intent IR, persistent typed memory, deterministic reproducible execution, and verifiable provenance. We proposed a controlled ablation and model-swap protocol to compare the marginal effects of architecture and scale. Finally, we presented Matrix as a construction-time case study: a small team hand-authored the memory substrate and specification, and substrate-bound agents produced a large multi-module system while preserving key invariants. The case study motivates the thesis; the protocol is how the thesis should be tested.

Reference Implementation

The source-available reference implementation is listed as: <https://github.com/paxlabs-inc/matrix-core>. If the repository or audit bundle is not publicly accessible at review time, independent readers should treat claims depending on inspection as provisional until access is provided.

References

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," arXiv:2001.08361, 2020.
- [2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Rutherford, D. de Las Casas, L. Hendricks, J. Welbl, A. Clark, et al., "Training compute-optimal large language models," arXiv:2203.15556, 2022.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," NeurIPS, 2023, arXiv:2302.04761.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," ICLR, 2023, arXiv:2210.03629.
- [5] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic AI," arXiv:2506.02153, 2025.
- [6] E. Y. Chang, "The missing layer of AGI: From pattern alchemy to coordination physics," arXiv:2512.05765, 2025.
- [7] A. Yagubyan, "How consistent are LLM agents? Measuring behavioral reproducibility in multi-step tool-calling pipelines," arXiv:2605.28840, 2026.
- [8] R. C. Merkle, "A digital signature based on a conventional encryption function," Advances in Cryptology - CRYPTO, 1987.
- [9] W3C, "Decentralized identifiers (DIDs) v1.0," W3C Recommendation, 2022.
- [10] Anthropic, "Model context protocol specification," 2024.
- [11] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., "A survey on large language model based autonomous agents," arXiv:2308.11432, 2023.
- [12] PaxLabs Inc., "Matrix Core: Source-available cognition and UX layer for the Paxeer Network," <https://github.com/paxlabs-inc/matrix-core>, 2026, v0.2.5.